

Docker API exposing to external network via port — Hands-On Tutorial

(WSL + Linux + Windows + macOS)

2/32

1. What is the Docker API (quick recap)

Docker Engine exposes a **REST API** used by:

- Docker CLI
- Docker Desktop
- Portainer, Jenkins, GitLab, Kubernetes
- Any HTTP client (`curl` , SDKs)

Default endpoint:

Unix socket: `/var/run/docker.sock`

Optional:

TCP **socket**: `tcp://HOST:2375` (NO TLS)
TCP **socket**: `tcp://HOST:2376` (TLS)

3/32

2. Check your current Docker API setup

Verify Docker Engine

```
docker version
docker info
```

Check socket

```
ls -l /var/run/docker.sock
```

Typical output:

```
srw-rw---- 1 root docker ...
```

This means:

- Root or `docker` group can access API
- Others cannot

4/32

3. Using Docker API locally (Unix socket)

Test API with curl (no TCP needed)

```
curl --unix-socket /var/run/docker.sock http://localhost/version
```

Expected JSON response:

```
{
  "Version": "24.x.x",
  "ApiVersion": "1.43",
  ...
}
```

More examples:

```
# List containers
curl --unix-socket /var/run/docker.sock http://localhost/containers/json

# List images
curl --unix-socket /var/run/docker.sock http://localhost/images/json
```

This works on:

- Linux
- WSL
- macOS
- Windows (via Docker Desktop VM)

4. Exposing Docker API over TCP (DANGEROUS if misused)

Why expose it?

- Remote CI/CD
- Portainer
- External automation tools

Why dangerous?

Docker API = root access to host

5. Expose Docker API on Linux / WSL (docker.io)

Step 1: Edit Docker daemon config

```
sudo nano /etc/docker/daemon.json
```

Add:

```
{
  "hosts": [
    "unix:///var/run/docker.sock",
    "tcp://0.0.0.0:2375"
  ]
}
```

Step 2: Restart Docker

use `systemctl` `service` or `kill` depending on OS

```
sudo systemctl daemon-reexec
sudo systemctl restart docker
```

if this gives error in wsl,

1 Edit Docker service override

```
sudo systemctl edit docker
```

Add

```
[Service]
ExecStart=
ExecStart=/usr/bin/dockerd
```

Save as `override.conf` usual location is
`/etc/systemd/system/docker.service.d/override.conf`

and exit and retry `sudo systemctl restart docker`

This removes `-H fd://.`

```
sudo service docker status
sudo service docker stop
sudo service docker start
```

```
sudo pkill dockerd
sudo dockerd &
```

Step 3: Verify port is open

```
ss -lntp | grep 2375
```

6. Test Docker API over TCP

From same machine:

```
curl http://localhost:2375/version
```

From another machine:

```
http://<host-ip>:2375/containers/json
```

Open URL in browser

<http://localhost:2375/containers/json> (*http://localhost:2375/containers/json*)

<http://localhost:2375/containers/json?all=true> (*http://localhost:2375/containers/json?all=true*)

If this works → Docker is remotely controllable.

7. Windows (Docker Desktop)

Docker Desktop behavior

- Docker runs **inside a VM**
- API is **not exposed by default**

Enable TCP API

1. Docker Desktop → Settings
2. Docker Engine
3. Edit JSON:

```
{
  "hosts": [
    "unix:///var/run/docker.sock",
    "tcp://0.0.0.0:2375"
  ]
}
```

1. Apply & Restart

Test (PowerShell)

```
curl http://localhost:2375/version
```

8/32

8. macOS (Docker Desktop)

Same as Windows:

- Docker → Settings → Docker Engine
- Add TCP host
- Restart

Test:

```
curl http://localhost:2375/version
```

9. Using Docker API programmatically

Example: Create container using curl

```
curl -X POST http://localhost:2375/containers/create \
-H "Content-Type: application/json" \
-d '{
  "Image": "nginx",
  "HostConfig": {
    "PortBindings": {
      "80/tcp": [{ "HostPort": "8080" }]
    }
  }
}'
```

Start it:

```
curl -X POST http://localhost:2375/containers/<container-id>/start
```

10. Securing Docker API (CRITICAL)

NEVER do this in production

```
tcp://0.0.0.0:2375
```

Anyone can:

- Run containers
- Mount
- Delete data
- Escape containers

11. Secure Docker API with TLS (Recommended)

Generate certs (simplified overview)

```
mkdir ~/docker-certs
cd ~/docker-certs
```

Create CA, server, client certificates Then update daemon.json:

```
{
  "hosts": ["tcp://0.0.0.0:2376"],
  "tls": true,
  "tlscacert": "/etc/docker/ca.pem",
  "tlscert": "/etc/docker/server-cert.pem",
  "tlskey": "/etc/docker/server-key.pem",
  "tlsverify": true
}
```

Client usage:

```
docker --tlsverify \
  --tlscacert=ca.pem \
  --tlscert=cert.pem \
  --tlskey=key.pem \
  -H=tcp://host:2376 ps
```

12/32

12. Safer Alternatives to TCP Exposure

Option 1: SSH tunneling (BEST)

```
ssh -L 2375:/var/run/docker.sock user@host
```

Then locally:

```
export DOCKER_HOST=tcp://localhost:2375
docker ps
```

Option 2: Portainer agent

- Uses encrypted tunnel
- No open Docker API

13/32

13. Common Issues & Fixes

Issue: **permission denied**

```
sudo usermod -aG docker $USER
newgrp docker
```

Issue: API works locally but not remotely

- Firewall blocking port
- Docker bound to `127.0.0.1`

Check:

```
ss -lntp | grep docker
```

Issue: Docker Desktop ignores daemon.json

- Docker Desktop overwrites config
- Must edit via UI, not file

14/32

14. How to Verify Exposure (Security Check)

```
nmap -p 2375 <host-ip>
```

If open → system is vulnerable.

Also test:

```
curl http://<host-ip>:2375/containers/json
```

If this works → **full host compromise possible**

15/32

15. Recommended Learning Flow

1. Unix socket API usage
2. TCP exposure locally
3. Remote API testing
4. TLS protection
5. SSH tunneling

6. CI/CD integration

7. Kubernetes vs Docker API comparison

16/32

Summary (Key Takeaways)

- Docker CLI = Docker API client
- Unix socket is safest
- TCP without TLS is **root-level vulnerability**
- SSH tunnel is simplest secure method
- Docker Desktop behaves differently from Linux/WSL

17/32

Optional Read

18/32

Restart docker daemon after updating configuration

```
/etc/docker/daemon.json
```

Why `systemctl` vs `service` vs `kill dockerd` Feels Random

It is **not random**. It depends on **who PID 1 is**.

Everything starts with **PID 1**.

19/32

1. The One Rule That Explains Everything

Run this **first, always**:

```
ps -p 1 -o pid,comm,args
```

This tells you **what controls the system**.

Possible outputs and meaning

Case A: systemd is PID 1

```
1 systemd /sbin/init
```

You are on:

- Real Linux
- OR WSL with systemd enabled
- OR VM

Correct commands:

```
systemctl restart docker  
systemctl status docker
```

`service` works only as a wrapper.

20/32

Case B: init / bash / sh is PID 1

```
1 init
```

or

```
1 bash
```

You are on:

- Old WSL
- Minimal container
- Chroot-like environment

Correct commands:

```
sudo service docker restart
```

If that fails:

```
sudo pkill dockerd  
sudo dockerd &
```

`systemctl` will **never** work here.

21/32

2. What Each Command Actually Is

`systemctl`

- Talks directly to **systemd**
- Requires:
 - systemd running
 - D-Bus
- **Modern, correct, strict**

If systemd isn't PID 1 → fails.

22/32

`service`

- Legacy compatibility layer
- Tries:
 - systemd (if present)
 - SysV init scripts
- Works in more places, less reliable

Think of it as:

“Do whatever you can to restart this service”

23/32

pkill dockerd

- Nuclear option
- Kills Docker daemon process
- You manually restart it

Works **anywhere**, but:

- No logging
- No dependency handling
- No auto-restart

Use only for debugging.

24/32

3. Why WSL Makes This Worse

WSL historically:

- Had **no init system**
- PID 1 was just a shell
- Services were fake

Now:

- Windows 11 supports **systemd**
- But it is **opt-in**
- Mixed environments exist

This creates:

- Tutorials that half-work
- Students with different behavior
- Inconsistent lab results

4. Decision Tree (Memorize This)

```

Start
|
|-- ps -p 1
|   |
|   |-- systemd
|   |   |
|   |   |--> systemctl restart docker
|   |
|   |-- NOT systemd
|   |   |
|   |   |--> service docker restart
|   |       |
|   |       |-- fails
|   |           |
|   |           |--> pkill dockerd && dockerd &
  
```

This is the **correct mental model**.

Now: macOS + Docker Desktop (Why `ss` Doesn't Work)

macOS is **not Linux**.

Even though Docker runs Linux containers, **your terminal is NOT inside that Linux VM**.

5. macOS Architecture (Critical Understanding)

```

macOS Terminal
|
| (docker CLI)
|
Unix socket (proxy)
|
Docker Desktop VM (Linux)
|
dockerd
  
```

So when you run:

```
ss -lntp
```

You are checking **macOS kernel**, not Linux VM.

macOS:

- Does not have `ss`
- Does not expose VM networking
- Does not show container ports internally

28/32

6. How to Troubleshoot Docker API on macOS

What DOES work on macOS

Check Docker daemon

```
docker info
```

Check API via socket

```
curl --unix-socket \
~/Library/Containers/com.docker.docker/Data/docker.sock \
http://localhost/version
```

Check TCP (if exposed)

```
curl http://localhost:2375/version
```

29/32

What will NEVER work on macOS host

```
ss -lntp
netstat -lntp
```

Because:

- The daemon is inside a VM
- Ports are proxied
- Host kernel doesn't see them

7. How macOS Users Should Debug Instead

Use Docker-native tools

```
docker ps
docker inspect
docker context inspect
```

Or enter the VM (advanced)

```
docker run --privileged -it --pid=host alpine sh
```

Inside that container:

```
ss -lntp
```

Now you are **inside the VM namespace**.

8. Why Docker Desktop Hides This On Purpose

Reasons:

- Avoid leaking VM internals
- Avoid kernel confusion
- Keep UX consistent

Docker Desktop is a **product**, not raw Linux.

9. Clean Summary Table

Environment	PID 1	systemctl	service	ss
Linux server	systemd	Yes	Yes	Yes
WSL (old)	bash/init	No	Sometimes	Maybe
WSL (new)	systemd	Yes	Yes	Yes
macOS host	launchd	No	No	No
Docker VM	systemd	Yes	Yes	Yes